

Mira Intel: Automated Damage Detection for Offshore Wind Turbine Inspection

15.572 Analytics Lab

Ayela Chughtai, Riya Parikh, Lauren Montigue, Barath Velmurugan

Mentor: Jerrold M. Grochow

December 11, 2025

Contents

1	Executive Summary	2
2	Introduction	3
3	Project Overview	3
4	Phase I: Agent Builder and MirAI Prototype	4
4.1	MirAI: A Multi-Agent Workflow for Structured Damage Assessment	4
4.2	Knowledge Engineering and Case Library Construction	5
4.3	Technical Constraints and Strategic Pivot	5
5	Phase II: Classical Computer Vision Pipeline Using YOLOv8	6
5.1	Dataset Overview	6
5.2	YOLOv8 Training Pipeline	7
5.3	Model Results and Evaluation	10
6	Phase 3: Interactive User Interface (Gradio)	12
7	Business Impact	13
7.1	Cost Delta for Severe Damage Types	13
7.2	Impact at Scale	14
8	Conclusion	14
	Appendix	16
	References	23

1 Executive Summary

Offshore wind operators rely on slow, hazardous, and inconsistent inspection workflows that depend on rope-access technicians or manually piloted drones. These methods require turbine shutdowns, produce variable image quality, and frequently miss early-stage blade damage, leading to avoidable failures and high repair costs. Mira Intel’s patented stabilized drone hardware enables high-quality imaging while turbines remain in motion, creating a foundation for safer, faster, and more scalable automated inspections.

Our Analytics Lab project focused on building the software layer needed to operationalize this vision. We developed a high-performing computer vision system capable of detecting and classifying turbine-blade defects directly from drone imagery. After evaluating multiple model configurations, we converged on a 7-class YOLOv8 detector enhanced through targeted oversampling and augmentation. The final model demonstrated strong overall performance and exceptional accuracy on safety-critical “severe” damage categories, including lightning strikes, oil leakage, leading-edge erosion, and surface cracks. When collapsed into a severe vs. non-severe damage, the system achieved an F1-score of 96.5%, with 96.4% precision and 96.7% recall, enabling highly reliable identification of the defects that matter most for turbine health and operator safety.

To support real-world evaluation, we built a lightweight Gradio interface that allows users to upload images, view annotated detections, and access class-frequency and confidence summaries. This tool enables immediate qualitative assessment and demonstrates a clear path toward field deployment.

The model’s performance translates into meaningful economic value for wind operators. Using industry repair cost deltas and our per-class recall, early detection provides an average of \$84,600 in avoided cost per severe incident. For a representative 100-turbine offshore wind farm, this corresponds to approximately \$135,000 in avoided annual repair costs and more than \$3.4 million in savings over a 25-year operating life.

Beyond these direct savings, this work establishes the software foundation for Mira Intel’s long-term roadmap. The most significant opportunity ahead is a real-time, on-drone inference, enabling instant damage assessment during flight, a capability that does not exist in today’s market and would materially differentiate the company’s inspection platform.

Overall, this project creates a pipeline whereby Mira Intel can pair its stabilized drone hardware with a robust automated perception system to materially reduce inspection costs, enhance technician safety, and enable predictive maintenance at fleet scale.

2 Introduction

Offshore wind turbines operate in some of the harshest environmental conditions, facing exposure to saltwater and soil corrosion, lightning strikes, and wildlife impact. To ensure safe and reliable power generation, turbine blades require regular inspection, but current inspection methods are risky, costly, and inefficient.

Traditional approaches rely on rope-access technicians who physically climb turbine structures, or on manually piloted drones that are difficult to stabilize in offshore wind conditions. Both require stopping the turbine, which leads to lost energy production. These labor and downtime constraints make large-scale condition monitoring slow, expensive, and operationally hazardous.

Mira Intel was founded to address these challenges by automating the inspection process end-to-end. Their patented hardware innovation equips drones with a stabilizing camera system capable of capturing crisp, high-resolution images of turbine blades while the turbine remains in motion. This eliminates the need for rope-access teams, reduces operational downtime, and generates consistent imaging data even in dynamic offshore environments.

On the software side, Mira Intel is developing computer vision and machine learning technologies that automatically detect, classify, and predict the deterioration of blade damage. By pairing stabilized drone imaging with automated analysis, the company aims to deliver a scalable, high-frequency inspection solution that improves safety, reduces maintenance costs associated with labor and downtime, and enables predictive maintenance for wind operators.

3 Project Overview

Our Analytics Lab project has focused on the software component of Mira Intel’s platform; more specifically, building a machine learning model capable of detecting and classifying damage on wind turbine blades from drone-captured images.

Using the image data provided by Mira Intel, our goal was to evaluate modern computer vision architectures and determine their potential to support automatic inspection. The objective was to develop a model that could most accurately identify severe damage categories such as surface cracks, erosion on the leading edge of the blade, and lightning strikes.

By testing computer vision models, our project creates a fully automated inspection pipeline in which machine learning systems can instantly analyze stable and high-quality drone images to support fast decision-making. The results of this work lay the foundation for predictive maintenance workflows, reduced manual labor, and more efficient oversight of offshore wind infrastructure.

4 Phase I: Agent Builder and MirAI Prototype

At the outset of the project, Mira Intel asked our team to explore whether OpenAI’s Agent Builder platform could serve as the foundation for a unified, intelligent wind-turbine inspection assistant. The company envisioned an agentic workflow that would not only assess damage from turbine images, but also interpret environmental context, reference expert maintenance protocols, and forecast damage progression over multiple time horizons. They provided a detailed project plan outlining the development of a custom agent capable of producing schema-validated JSON outputs, generating calibrated risk forecasts, citing domain knowledge, and supporting downstream decision-making. In alignment with this direction, our team spent the first half of the project designing, building, and evaluating a multi-agent system within Agent Builder. Namely, MirAI, which we presented during our Midpoint Deliverable.

4.1 MirAI: A Multi-Agent Workflow for Structured Damage Assessment

Working within Agent Builder, we implemented a modular, multi-agent architecture designed to replicate the intended end-to-end workflow of an automated turbine inspection assistant. As shown in our midterm architecture diagram, MirAI consisted of several coordinated components:

- **Perception Agent:** Intended to interpret turbine images and produce shortlists of likely damage types.
- **Contextual Intelligence Agents:** Responsible for integrating environmental conditions, turbine model information, prior maintenance history, and material properties into the assessment pipeline.
- **Reasoning Enhancement Agent:** Designed to refine the interpretation of cues, resolve inconsistencies, and synthesize evidence from multiple sources.
- **Temporal Forecasting Agent:** Tasked with projecting damage progression and maintenance windows at 30/90/180/365-day horizons.
- **Serialization Agent:** Responsible for producing schema-valid JSON outputs including probabilities, rationales, assumptions, and citations.

We were able to prototype the full reasoning, retrieval, forecasting, and output-validation pipeline. MirAI successfully handled multi-step logic, domain-conditioned reasoning, and multi-agent collaboration, demonstrating the potential of agentic workflows for turbine inspection.

4.2 Knowledge Engineering and Case Library Construction

A significant component of Phase I involved building the knowledge infrastructure MirAI would rely on. Using expert case studies as training data fed into the knowledge vector, it was able to understand blade erosion mechanisms, crack progression models, environmental accelerants such as salt spray and freeze-thaw cycles, inspection intervals, and uncertainty drivers.

By the project midpoint, MirAI could ingest case files, retrieve relevant knowledge, produce rationales with citations, and generate consistent JSON outputs that reflected the system’s reasoning. Although the image components were simulated rather than rendered from real turbine photographs, the conceptual pipeline for a fully integrated assistant was established.

4.3 Technical Constraints and Strategic Pivot

During implementation, however, we identified fundamental constraints in the updated version of OpenAI’s Agent Builder. Specifically, it did not support custom vision models. As a result, MirAI could reason about damage descriptions, environmental conditions, and progression dynamics, but it could not perform *direct visual damage detection*, which was essential to the core inspection workflow.

This limitation led us to conclude that a classical computer vision model would be required to achieve reliable, image-based damage detection. The agentic pipeline we built, while valuable for qualitative reasoning and structured assessment, could not function independently of a dedicated perception model trained on real turbine imagery.

Despite the pivot, Phase I provided value to the final outcome of the project. It clarified the role of environmental modifiers, domain knowledge, and long-horizon forecasting in turbine assessment. It established schema standards and reasoning requirements that informed our eventual computer vision model. It also produced a clear set of requirements for future integration of agentic workflows once vision model support becomes available on the Agent Builder platform. Most importantly, it ensured that the transition to classical computer vision was grounded in a thorough understanding of Mira Intel’s broader technical and operational goals.

This foundation shaped Phase II, where we refocused efforts to build a high-performance damage-classification model capable of operating directly on images of turbine damage.

5 Phase II: Classical Computer Vision Pipeline Using YOLOv8

5.1 Dataset Overview

Mira Intel provided a multiclass dataset of drone-captured turbine blade images annotated with eight damage categories. The eight annotated classes in the dataset are: Drain Hole Impairment, Lightning Strike, Oil Leakage, PU-tape (protective coating), Paint, Surface Crack, Dirt, and Leading-edge (LE) erosion. Each image may contain zero, one, or multiple instances of damage; and multiple images may depict the same turbine surface under different orientations. Occasionally, there are also duplicate images with identical labels across both the main and auxiliary datasets.

This duplication generally arises from natural multi-label damage scenarios. As a consequence, the dataset contains true biological duplicates (same photo, same damage) and geometric duplicates (image transformations). Without careful splitting, such duplicates can leak across train and test sets, artificially inflating accuracy. To mitigate this, we ensured there was no leakage between similar images across the train, validation, and test splits. After pre-processing, our data was disaggregated as follows:

Split	Drain Hole	Lightning	Oil Leak	PU-tape	Paint	Surface Crack	Dirt	LE-erosion	Total
Train	6	312	720	280	1203	603	552	1912	5588
Validation	0	235	203	2	170	204	35	280	1129
Test	1	3	14	12	90	5	49	128	302
Total	7	550	937	294	1463	812	636	2320	7019

Table 1: Distribution of labeled objects across train, validation, and test splits for all eight classes.

The distribution in Table 1 reveals an unusual but strategically valuable property of the dataset¹: high-severity damage classes (LE-Erosion, Surface Crack, Lightning Strike) constitute the slight majority of labeled objects. In total, severe classes account for 3,682 of the 7,019 labeled instances (52.5%). In particular: Leading-edge Erosion (class 7) is the most common class and makes up 33.1% of all labels, Surface Crack (class 5) is 11.6%, and Lightning Strike (class 1) is 7.8%.

These three severe classes are the damage types that Mira Intel considers most operationally critical as they correspond to structural or aerodynamic failures that may require immediate inspection, repair, or blade replacement. The fact that these classes collectively form the majority of the dataset is not representative of real-world turbine distributions where severe damage is rare compared to superficial conditions like dirt or chipped paint. However, it is highly beneficial for model development as it is significantly less costly to incorrectly predict severe damage than to incorrectly predict non-severe damage.

In contrast, the five non-severe classes make up a smaller portion of the dataset: Paint is 20.8%,

¹The counts reflect the merged main + auxiliary dataset after harmonizing label names. Minor fluctuations may occur from annotation updates

Dirt is 9.1%, Oil Leakage is 13.3%, PU-tape is 4.2% and Drain Hole Impairment is just 0.1%. These categories represent minor surface conditions or cosmetic degradations that generally do not pose immediate operational risk. Their lower prevalence reduces the chance that the model will overfit to trivial visual patterns instead of learning meaningful structural defects.

Overall, the dataset’s composition, dominated by high-severity classes despite their real-world rarity, reflects the business priority of building an inspection pipeline that emphasizes safety-critical damage detection above all else. This distribution provides robust supervision for the classes that have the greatest impact on turbine reliability, maintenance scheduling, and cost.

Ultimately, the computer vision model we used needed to support a multivalent labeling structure, which led us to choose YOLOv8. The other structural idiosyncrasies of the data, such as class imbalance, motivated other modeling techniques such as oversampling and targeted hyperparameter tuning.

5.2 YOLOv8 Training Pipeline

The YOLOv8 object detection framework served as the backbone of our computer vision pipeline. YOLOv8 offers strong performance on small to medium-scale datasets, built-in support for multi-class detection in a single forward pass, an efficient deployment footprint suitable for future on-drone or edge inference, and robust default training recipes including automatic learning rate scaling and rich data augmentation.

We used the YOLOv8n (nano) variant initialized from COCO-pretrained weights for faster training. Unless otherwise noted, training runs used YOLO’s default optimizer and scheduling with $\text{imgsz} = 640$, $\text{batch size} = 16$, $\text{epochs} = 30$, $\text{auto-learning-rate} = \text{heuristic}$ (no manual LR tuning), and standard YOLOv8 augmentations (horizontal flips, HSV jitter, random perspective, cutout/erasing, and RandAugment). YOLOv8 normalizes gradients internally to emulate larger batch sizes, so a relatively small batch of 16 did not materially limit convergence or stability.

Model 1: To start, we employed a baseline model on the raw dataset to understand the performance ceiling of the YOLOv8n design. The model was trained on all 8 classes with no data balancing or other techniques (done later). The results confirmed that extreme class imbalance was causing poor overall detection/recall for rare but business-critical damages such as Lightning Strike and Surface Crack. Thus, our data, as given, was not appropriate for training. (see Appendix A)

Model 2: The second model, YOLOv8s (better than YOLOv8n), was run on an oversampled dataset. Oversampling was achieved by duplicating images of minority classes by 10 to match the frequency of the majority classes. Although there was slight improvement, the oversampling approach ultimately failed as Model 2 still struggled on minority classes. (see Appendix B)

Model 3: To better understand the model’s behavior under controlled conditions, we constructed a small balanced subset by undersampling the majority classes to obtain approximately equal numbers of images per class, with the exception of the Drain Hole class, which still had only seven images. We then retrained YOLOv8n on this subset with more epochs and a higher input resolution (increasing from 512 to 640). The goal was diagnostic rather than production-ready performance: we wanted to confirm that the model architecture itself could learn all classes when given a balanced, simplified training distribution. This experiment demonstrated that YOLOv8n could indeed fit all classes on a small, balanced set, suggesting that the earlier failure was driven primarily by data distribution rather than architectural limitations. (see Appendix C)

Model 4: A major improvement in model performance came when Mira shared an additional open-source dataset containing box-bounded wind turbine damage. With the 8-class baseline for the larger dataset, our baseline model converged quickly in terms of loss, but class-level performance remained weak. In particular, Lightning Strike and Paint detections were highly unstable, and predictions showed a strong bias toward the most frequent category, Leading-edge Erosion. (see Appendix D)

Model 5: Because there was an emphasis on safety-critical damage, we next experimented with a reduced 5-class formulation in which we filtered out the lowest-priority surface classes (Paint and Dirt), as well as the least-occurring class (Drain Hole Damage). On this reduced-label configuration, YOLOv8n exhibited noticeably better performance on the retained classes. However, this came at the cost of practicality and robustness: excluding common surface-level conditions simplified the task in a way that would not match real-world inspections where the model would be expected to distinguish severe damage from benign artifacts such as dirt and paint chipping. Therefore, these 5-class experiments were treated as a proof-of-concept rather than a direction for our final modeling. This validated that the model could learn essential patterns under more controlled conditions, and progressively refining the sampling strategy would lead to further downstream improvements. (see Appendix E)

Model 6: Since Drain Hole Impairment appeared only seven times (0.1%) across the entire dataset, we removed this class from both training and evaluation. In this model, we used oversampling with augmentation to rebalance the training set. For each class, we constructed a list of images containing that label and repeatedly sampled from minority classes until all classes were represented at roughly the same frequency. For image augmentation, we took advantage of YOLOv8’s built-in augmentation pipeline, which applies techniques such as mosaic, mixup, horizontal flips, color and HSV jitter, scaling/zoom, translation, and light perspective transforms during training. For this model, we performed an evaluation not just on a class-by-class basis, but also looking at confusion matrices between severe and non severe classes to ensure maximum detection of severe damage. This oversampling with augmentation strategy yielded the best and most robust model.

Model 7: To test an extreme scenario, we trained on our reduced, 5-class set using undersampling

and focusing primarily on severe and semi-severe damage. With this configuration, the model achieved exceptionally high precision and recall ($> 99\%$) on severe damage. However, this came at the expense of robustness to more benign conditions. (see Appendix F)

Model 8: As a final stress test, we performed oversampling with augmentation for balanced classes while dropping three classes (Paint, Dirt, and Drainhole). This also yielded nearly perfect recall for severe classes on held-out data, but was similarly lacking in robustness. (see Appendix G)

Hyperparameter tuning was conducted on a reduced subset of the dataset due to the long training time required for our full-scale Model 6. We used Model 6’s architecture and settings as the baseline during tuning. On the smaller subset, none of the tuned configurations outperformed YOLOv8’s default hyperparameters, which are automatically selected based on dataset characteristics (e.g., image shapes, class distribution, and training-stable batch sizes). In computer vision, suboptimal hyperparameters found on a small proxy dataset do not necessarily translate to poor performance on the full model, but they still provide some directional signal. Given that YOLO’s built-in heuristics are well-tested and optimized for general performance, it is not surprising that our manual tuning did not outperform the auto-determined defaults, so we did not further test manual values on our full-scale model.

Summary of the hyperparameter tuning that was key for the best performing model (Model 6):

- **Subsets:** With computer vision models based on neural networks like YOLOv8, tuning on a smaller dataset does not necessarily generalize to a larger dataset and so all tuning was done using the full dataset.
- **Image size:** We increased the image size from 512 to 640, which produced consistent gains in mAP, especially for small and thin structures such as cracks and localized erosion.
- **Epochs:** YOLOv8 converged rapidly under the oversampled configuration: loss curves and validation metrics stabilized around epochs 18–20. We set epochs = 30 to allow stable convergence without overfitting.
- **Learning rate:** We retained YOLO’s automatic learning rate selection. This heuristic scales with model size, batch size, and dataset characteristics and performed well in practice; manual LR tuning did not yield consistent improvements in early experiments.
- **Augmentation:** We relied on YOLO’s default augmentation stack (flips, HSV jitter, perspective transforms, erasing, RandAugment). These operations, together with oversampling, effectively increased the diversity of minority-class examples without artificially changing the underlying label distribution.
- **Batch Size:** Batch size was limited by GPU memory. YOLOv8 normalizes gradients to emulate a large batch size, so no material impact on stability or accuracy

These results indicate that the final YOLOv8 model is both well-calibrated to Mira Intel’s business priorities and robust enough to form the core of an automated blade damage detection pipeline.

5.3 Model Results and Evaluation

The final and best-performing model in our pipeline was the 7-class YOLOv8n detector trained with oversampling and extensive augmentation, with the Drain Hole class removed due to extremely low support and inconsistent annotations. This configuration produced the strongest and most stable performance across all classes, particularly for the safety-critical damage categories that Mira Intel considers operationally high priority.

Figure 1 visualizes the evolution of the loss components (box loss, class loss, DFL loss) and performance metrics (precision, recall, mAP) across training. All losses decrease smoothly and consistently for both training and validation sets, with mAP stabilizing between epochs 18–22. This is a typical convergence pattern for YOLO models.

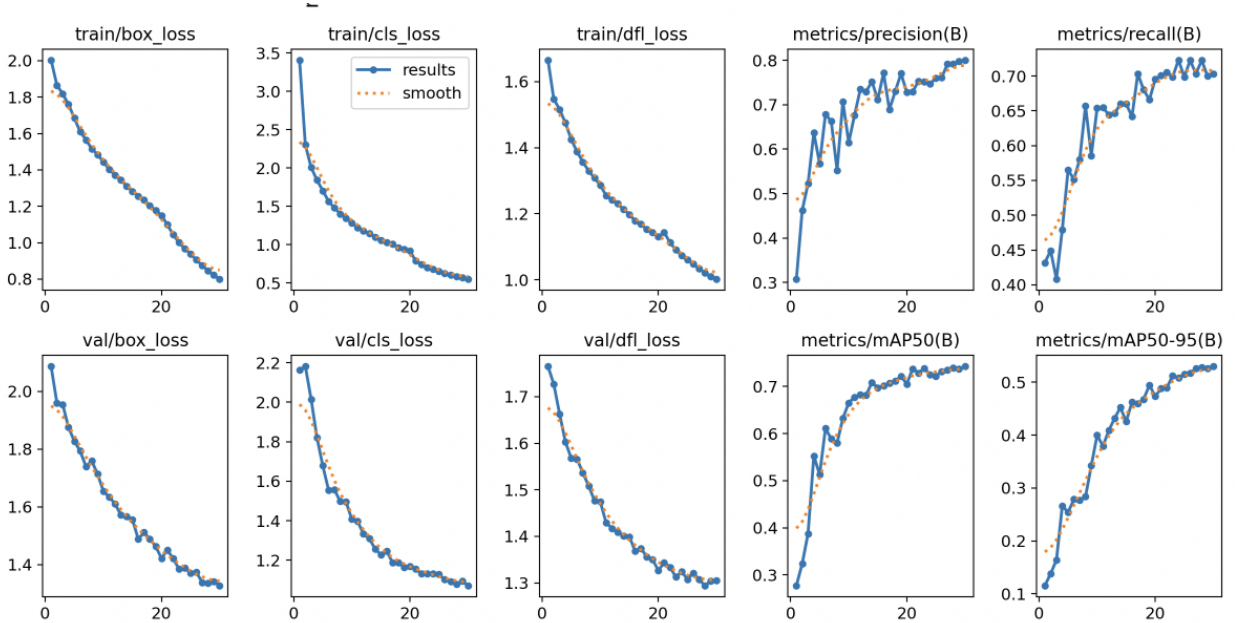


Figure 1: Training and validation loss curves and performance metrics for the final YOLOv8 model.

The absence of divergence between training and validation losses indicates no measurable overfitting. The model generalizes well despite augmentation and class rebalancing. Evaluation on the held-out test set (875 images, 1129 labeled objects) yielded: $mAP_{50} = 0.742$, $mAP_{50-95} = 0.531$, Precision = 0.799, Recall = 0.702.

The full set of class-level metrics is reported in Table 2.

Class	Precision	Recall	mAP@50	mAP@50-95
Lightning Strike	0.960	0.877	0.913	0.733
Oil Leakage	0.900	0.843	0.911	0.723
PU-tape	0.821	1.000	0.995	0.895
Paint	0.567	0.371	0.443	0.206
Surface Crack	0.859	0.500	0.601	0.381
Dirt	0.638	0.554	0.504	0.245
Leading-edge Erosion	0.851	0.768	0.834	0.533

Table 2: Per-class performance metrics for the final oversampled+augmented YOLOv8 model.

Severe damage categories (Lightning Strike, Oil Leakage, Surface Crack, and Leading-edge Erosion) consistently outperform non-severe classes in both detection quality and stability, reflecting their higher visual consistency and better representation after oversampling.

The full 8×8 confusion matrix is shown in Figure 2. Severe damage types exhibit strong isolation, with high true-positive counts and minimal cross-class confusion. Misclassifications primarily occur among non-severe classes (Paint, Dirt), which are visually noisy and less distinct structurally.

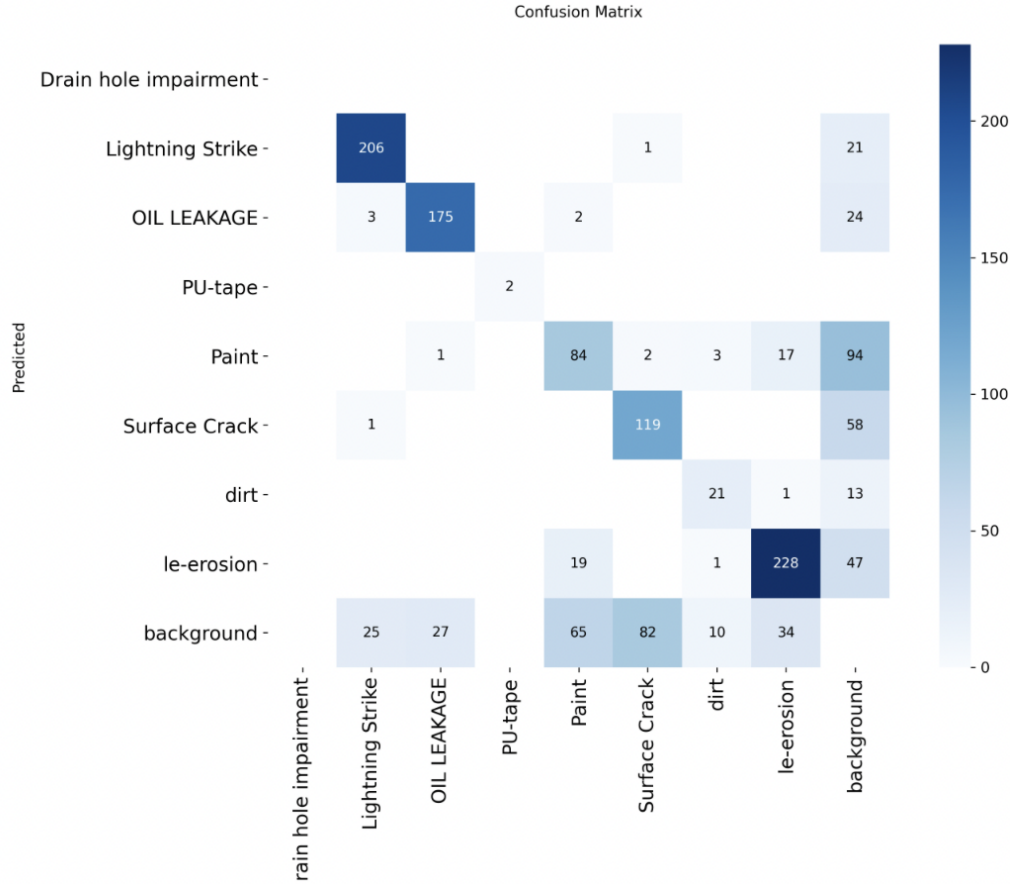


Figure 2: Confusion matrix for the final YOLOv8 model on the validation set.

Lightning Strike and Leading-edge Erosion achieve especially strong separability, with large, pure diagonal blocks indicating reliable detection and low false-positive rates.

For operational deployment, it is crucial that the model reliably distinguishes between severe structural hazards from non-severe cosmetic issues. We collapse the eight classes into two categories: Severe: Lightning Strike, Oil Leakage, Surface Crack, LE-Erosion and Non-Severe: Paint, Dirt, PU-tape. On an instance level, $\text{Precision}_{\text{severe}} = 0.964$, $\text{Recall}_{\text{severe}} = 0.967$, $\text{F1}_{\text{severe}} = 0.965$.

	Pred Severe	Pred Non-Severe
True Severe	556	19
True Non-Severe	21	288

The model therefore almost never misses truly severe damage and rarely misclassifies benign conditions as hazardous. This safety profile is ideal for automated inspection pipelines, where the cost of missing severe damage is substantially higher than the cost of flagging a benign issue.

6 Phase 3: Interactive User Interface (Gradio)

To make the final model accessible and visually interpretable, we developed an interactive web interface using Gradio. The interface loads the trained YOLOv8 model through a robust path-resolution function. Once the model is loaded, users may upload any turbine image, and the system runs inference with fixed confidence and IoU thresholds. The UI returns two outputs side-by-side: (i) an annotated image showing all detected bounding boxes and (ii) a text summary counting occurrences of each supported class (Lightning Strike, Oil Leakage, PU-tape, Surface Crack, and Leading-edge Erosion). This lightweight interface enables rapid model inspection without additional infrastructure and provides an intuitive foundation for future deployment.

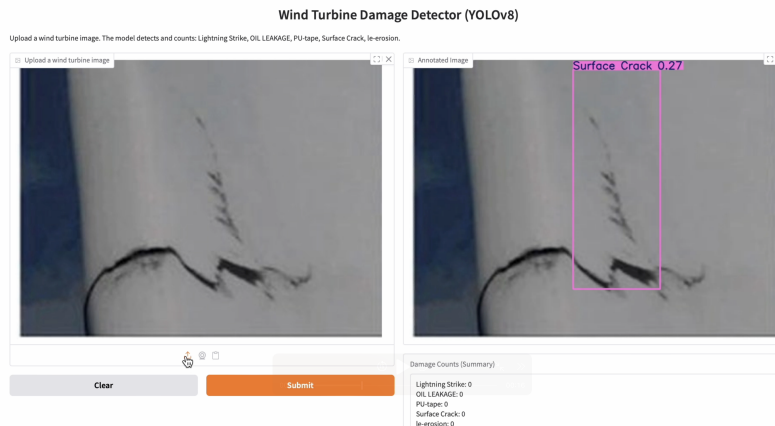


Figure 3: Gradio-based interactive interface for turbine damage detection.

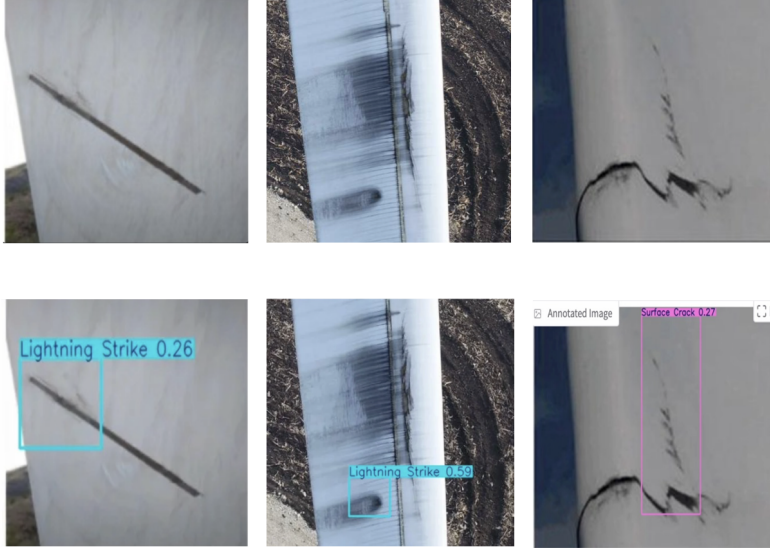


Figure 4: Model’s annotations on test images.

7 Business Impact

To complement the technical performance of our final YOLOv8 model, we quantify the operational and financial value of early detection of severe turbine-blade damage. Repair studies (e.g., Mishnaevsky 2019; Gurit; Belzona; NREL; Polytech) consistently show that late-stage failures can cost an order of magnitude more than early intervention. The *Cost Delta* (ΔC) represents the maximum avoidable cost when damage is identified early.

7.1 Cost Delta for Severe Damage Types

Cost Delta for Severe Damage Types

Damage Type	Cost Delta (ΔC)
Oil Leakage	\$200,000
Lightning Strike	\$170,000
Leading-edge Erosion	\$20,000
Surface Crack	\$10,000

Using the model’s per-class recall r_c , the recoverable savings per incident are given by

$$\text{Savings} \approx r_c \times \Delta C.$$

Oil Leakage:	168,600
Lightning Strike:	149,090
Leading-edge Erosion:	15,360
Surface Crack:	5,000

Assuming manual inspectors detect early-stage damage with probability $r_0 = 0.40$, the incremental value provided by our model is:

$$\text{Incremental Value} \approx (r_c - r_0) \times \Delta C.$$

Lightning Strike:	81,090
Oil Leakage:	88,600
Leading-edge Erosion:	7,360
Surface Crack:	1,000

On average, this yields approximately \$84,600 in avoided repair cost per severe incident.

7.2 Impact at Scale

Industry statistics indicate that 1.5–1.7% of offshore turbines experience a severe blade-level incident annually. For a representative 100-turbine farm, this corresponds to roughly 1.6 incidents per year. Applying our model’s average incremental value yields expected annual savings of approximately \$135,000, or about \$3.4 million over a 25-year operational life.

Mira Intel’s stabilized drone platform also eliminates turbine shutdown during imaging. Avoided downtime is valued at approximately \$73 per turbine per 15-minute inspection, scaling to \$877 across the Northeast fleet and nearly \$1M globally per inspection cycle.

Together, Mira Intel’s hardware and our high-accuracy perception model enable earlier interventions, reduced repair costs, and more frequent, no-downtime inspections, creating a scalable pathway to improved turbine reliability and lower lifetime maintenance costs.

8 Conclusion

In this project, we developed a full end-to-end damage detection pipeline for wind turbine blade inspection, beginning with data auditing and preprocessing, followed by multiple rounds of YOLOv8

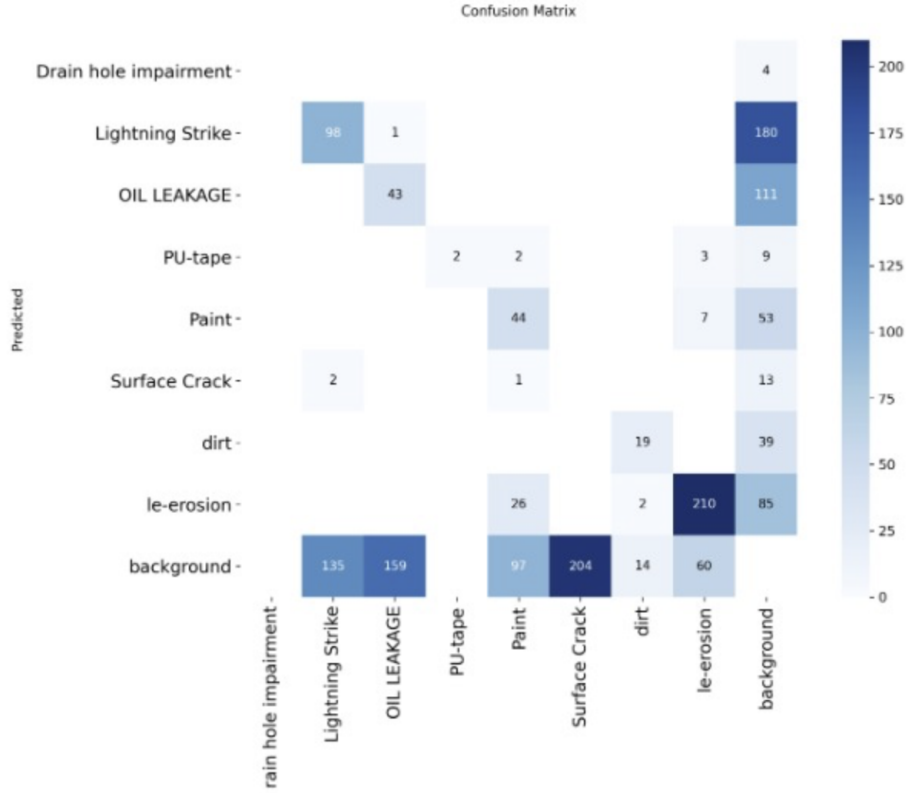
experimentation (including baselines, class-dropping, undersampling, oversampling, and targeted augmentation), before converging on a balanced, oversampled model with strong per-class accuracy and exceptional performance on safety-critical “severe” damage types. Through systematic evaluation, we met the core objectives of building a robust detector, understanding model error modes, and delivering a deployable interface for real-world testing.

Beyond technical performance, our model delivers meaningful operational value: early detection provides an average of \$84,600 in avoided repair cost per severe incident, translating to roughly \$137,000 in annual savings for a typical 100-turbine farm and more than \$3.4 million over a 25-year operational life.

Looking forward, the most promising opportunity for Mira Intel is real-time drone integration, enabling on-drone inference and immediate damage alerts during flight, a capability that would materially differentiate Mira Intel in the offshore wind market. Complementary avenues include expanding the dataset with more diverse turbine conditions, incorporating segmentation masks for geometry-aware modeling, leveraging larger vision backbones (e.g., YOLOv9, RT-DETR, SAM-based hybrids), ensuring temporal consistency across sequential blade frames, and automating maintenance report generation. Together, these extensions move the system closer to a fully operational, scalable, and industry-leading turbine health monitoring platform.

Appendix²

A Model 1 Metrics



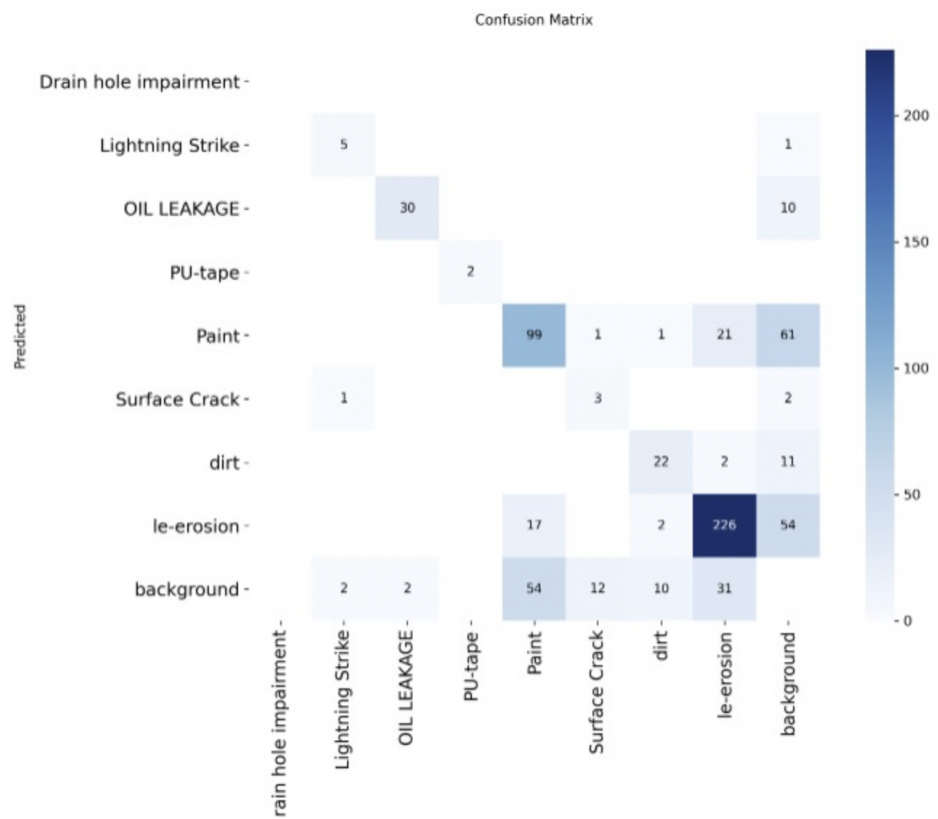
Model 1: Confusion matrix

Class	Precision	Recall	mAP@50	mAP@50-95
Lightning Strike	0.427	0.264	0.249	0.0796
OIL LEAKAGE	0.273	0.094	0.110	0.0411
PU-tape	0.566	1.000	0.828	0.419
Paint	0.526	0.176	0.227	0.0735
Surface Crack	0.000	0.000	0.0084	0.00225
dirt	0.551	0.514	0.486	0.208
le-erosion	0.790	0.577	0.672	0.309

Model 1: Per-Class Performance Metrics)

²Drain Hole Impairment is not depicted in tables due to all associated metrics being virtually 0.

B Model 2 Metrics

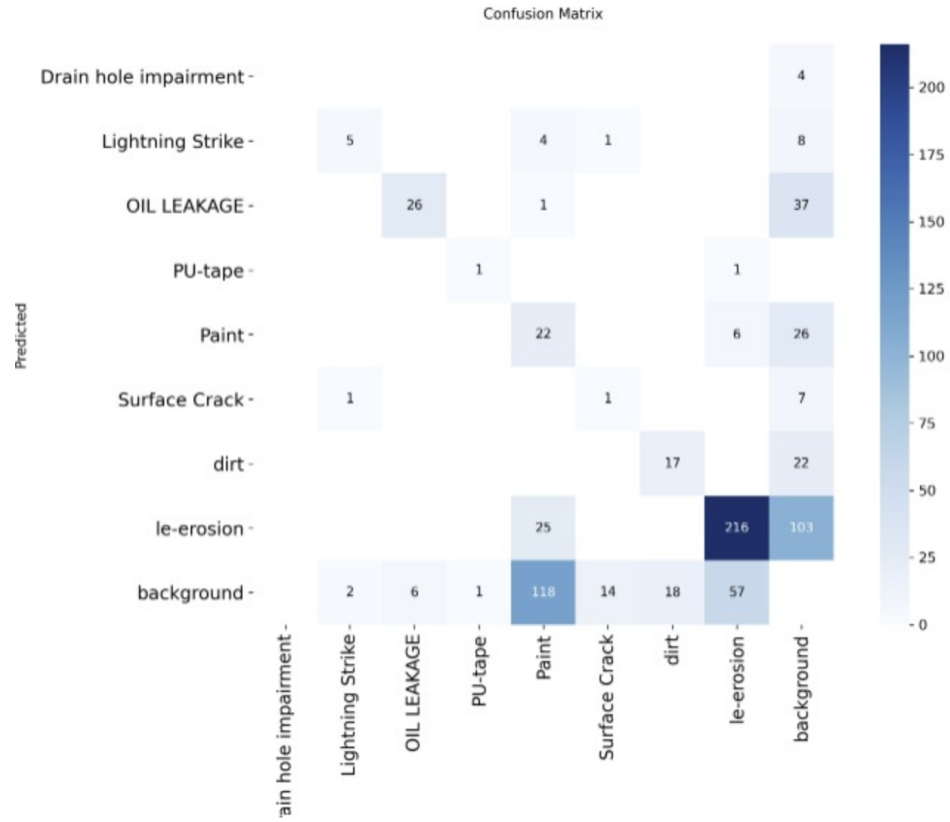


Model 2: Confusion matrix

Class	Precision	Recall	mAP@50	mAP@50-95
Lightning Strike	0.790	0.625	0.675	0.459
OIL LEAKAGE	0.818	0.906	0.898	0.667
PU-tape	0.859	1.000	0.995	0.921
Paint	0.583	0.476	0.492	0.211
Surface Crack	0.487	0.188	0.213	0.066
dirt	0.811	0.614	0.663	0.312
le-erosion	0.842	0.804	0.843	0.552

Model 2: Per-Class Performance Metrics)

C Model 3 Metrics

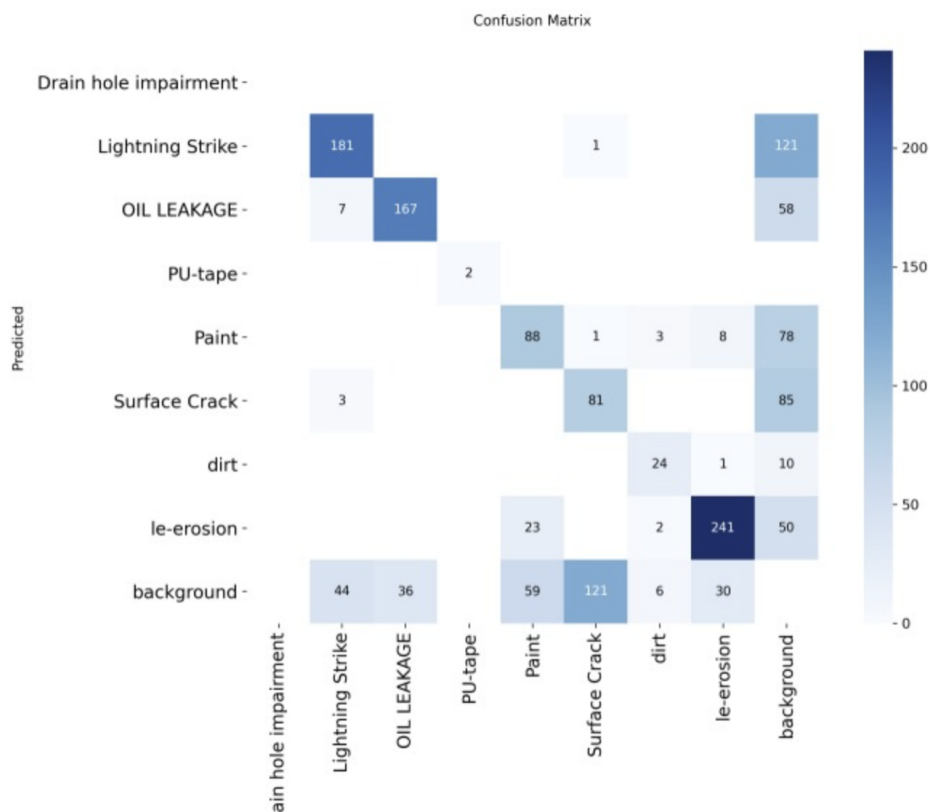


Model 3: Confusion matrix

Class	Precision	Recall	mAP@50	mAP@50-95
Lightning Strike	0.268	0.625	0.251	0.121
OIL LEAKAGE	0.384	0.781	0.521	0.240
PU-tape	0.347	0.500	0.695	0.423
Paint	0.412	0.124	0.160	0.0499
Surface Crack	0.099	0.063	0.0698	0.0161
dirt	0.431	0.486	0.426	0.166
le-erosion	0.640	0.782	0.683	0.329

Model 3: Per-Class Performance Metrics)

D Model 4 Metrics

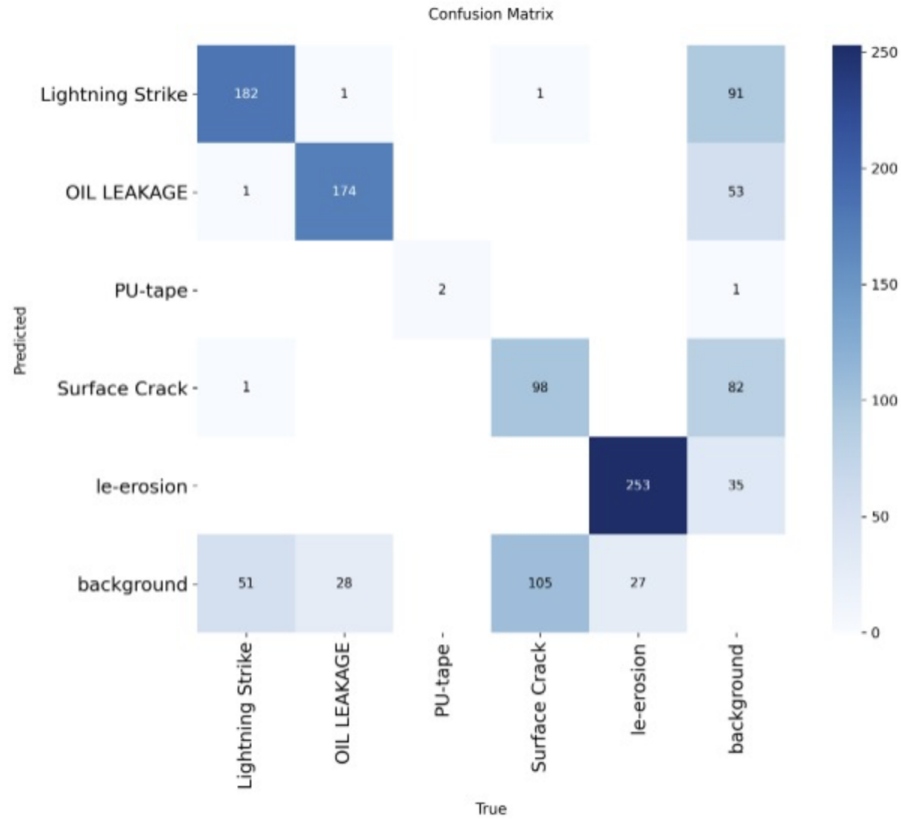


Model 4: Confusion matrix

Class	Precision	Recall	mAP@50	mAP@50-95
Lightning Strike	0.742	0.728	0.763	0.381
OIL LEAKAGE	0.761	0.783	0.854	0.476
PU-tape	0.720	1.000	0.995	0.746
Paint	0.540	0.380	0.456	0.205
Surface Crack	0.623	0.300	0.364	0.166
dirt	0.713	0.657	0.681	0.321
le-erosion	0.819	0.857	0.844	0.550

Model 4: Per-Class Performance Metrics)

E Model 5 Metrics

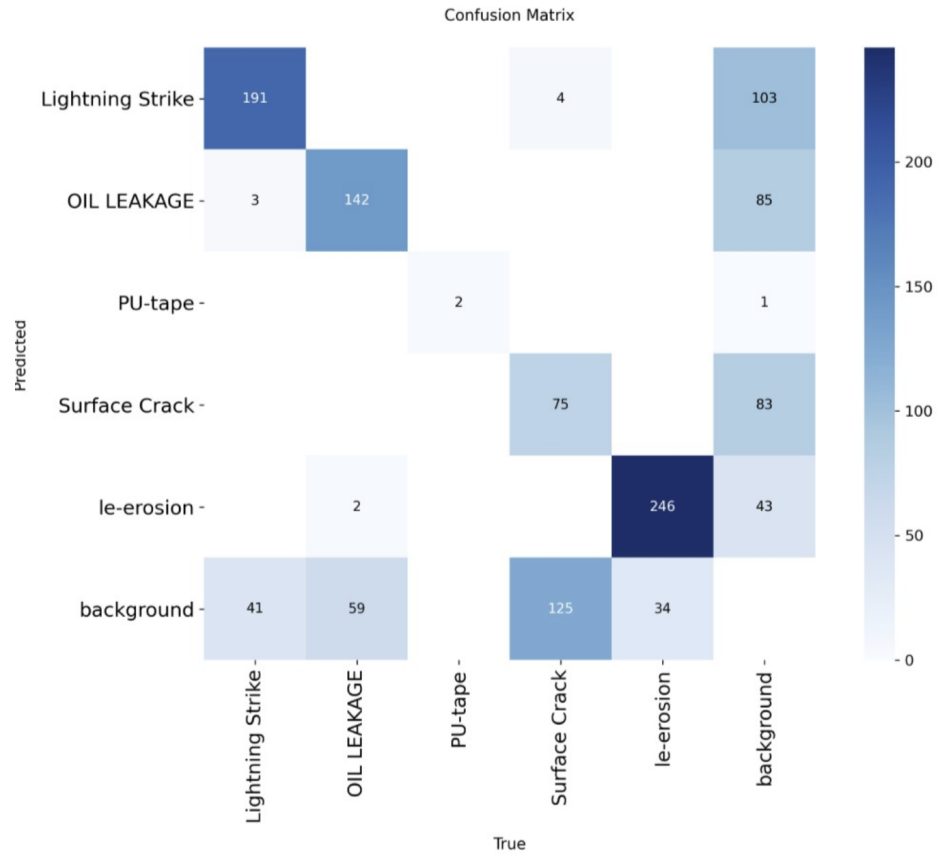


Model 5: Confusion matrix

Class	Precision	Recall	mAP@50	mAP@50–95
Lightning Strike	0.731	0.745	0.763	0.392
OIL LEAKAGE	0.779	0.823	0.858	0.491
PU-tape	0.549	1.000	0.995	0.821
Surface Crack	0.612	0.382	0.437	0.207
le-erosion	0.885	0.880	0.900	0.575

Model 5: Per-Class Performance Metrics)

F Model 7 Metrics

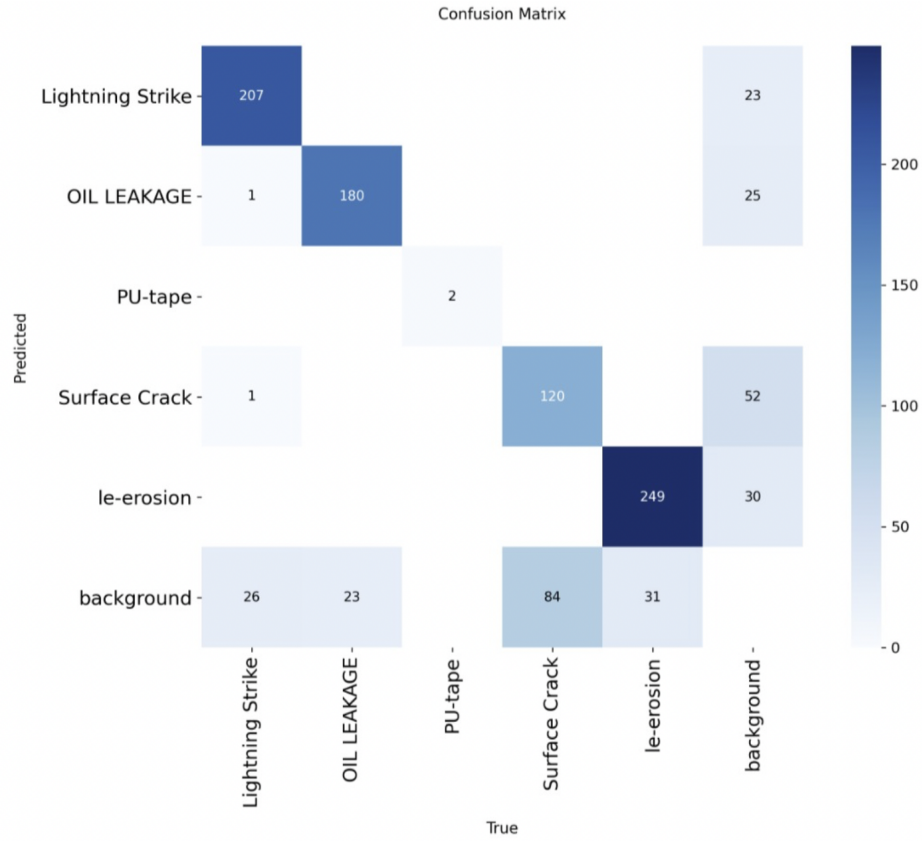


Model 7: Confusion matrix

Class	Precision	Recall	mAP@50	mAP@50–95
Lightning Strike	0.852	0.684	0.767	0.383
OIL LEAKAGE	0.822	0.544	0.677	0.344
PU-tape	0.901	1.000	0.995	0.646
Surface Crack	0.716	0.201	0.335	0.149
le-erosion	0.898	0.871	0.828	0.466

Model 7: Per-Class Performance Metrics)

G Model 8 Metrics



Model 8: Confusion matrix

Class	Precision	Recall	mAP@50	mAP@50-95
Lightning Strike	0.955	0.860	0.897	0.738
OIL LEAKAGE	0.924	0.852	0.914	0.723
PU-tape	0.890	1.000	0.995	0.895
Surface Crack	0.808	0.544	0.583	0.399
le-erosion	0.932	0.830	0.902	0.568

Model 8: Per-Class Performance Metrics)

References

- Mishnaevsky Jr., L. (2019). *Repair of wind turbine blades: Review of methods and related problems*. Renewable Energy.
<https://www.sciencedirect.com/science/article/abs/pii/S0960148119304252>
- Mishnaevsky Jr., L. & Thomsen, K. (2019). *Costs of repair of wind turbine blades*.
<https://docs.wind-watch.org/blade-repair-costs.pdf>
- Gurit. *Efficient wind turbine repairs / Gurit Shape article*.
<https://www.gurit.com/wp-content/uploads/bsk-pdf-manager/2022/08/Efficient-wind-turbine-repairs.pdf>
- PES. *The growing need for efficient in-field wind turbine blade repairs*.
<https://pes.eu.com/exclusive-articles/the-growing-need-for-efficient-in-field-wind-turbine-blade-repairs>
- Polytech. *Leading edge erosion white paper with site specific LEP solutions*.
<https://www.polytech.com/insights/leading-edge-erosion-white-paper-with-site-specific-lep-solutions/>
- State of Green / Wood Mackenzie. *Ensuring wind blade durability with leading edge protection*.
<https://stateofgreen.com/en/solutions/effective-and-resilient-leading-edge-protection/>
- Indji Watch. *Lightning strikes to wind turbines and the financial impact*.
<https://www.indjiwatch.com/hubfs/pdfs-clean/WP-lightening-strike-w.pdf>
- Raycap. *Wind turbine lightning protection*.
<https://www.raycap.com/wind-turbine-lightning-protection/>
- STLE. *Bearing and gearbox failures: Challenge to wind turbines*.
https://www.stle.org/files/TLTArchives/2020/08_August/Feature.aspx
- Shell. *Improving wind turbine gearbox reliability (white paper)*.
https://www.shell.com/business-customers/lubricants-for-business/products/wind-power/_jcr_content/root/main/containersection-0/simple/promo/links/item0.stream/1669133187184/d93d9778d1b9923e8e00850726d2c14a8d5931c0/wind-power-white-paper-low.pdf
- Wind Systems Magazine. *Monitoring for Maintenance*.
https://www.windsystemsmag.com/wp-content/uploads/pdfs/Articles/2009_May_June/TurningPoint0609.pdf

- Tamarindo. *Wind farm owners: How to cut the impact of gearbox failure*.
<https://tamarindo.global/insight/analysis/wind-turbine-gearbox-failure/>
- Zipdo. *Wind turbine failure statistics*.
<https://zipdo.co/wind-turbine-failure-statistics/>
- Herring, R. et al. (2019). *The increasing importance of leading edge erosion and a review of existing protection solutions*. Renewable and Sustainable Energy Reviews.
<https://www.sciencedirect.com/science/article/pii/S1364032119305908>